Generalized Origin-Destination-Time Flow Patterns

Chrysanthi Kosyfaki
Department of Computer Science
The University of Hong Kong
kosyfaki@cs.hku.hk

Nikos Mamoulis
Department of Computer Science and Engineering
University of Ioannina
nikos@cs.uoi.gr

Reynold Cheng
Department of Computer Science
The University of Hong Kong
ckcheng@cs.hku.hk

Ben Kao
Department of Computer Science
The University of Hong Kong
kao@cs.hku.hk

ABSTRACT

Analyzing flow of objects or data at different granularities of space and time can unveil interesting insights or trends. For example, transportation companies, by aggregating passenger travel data (e.g., counting passengers travelling from one region to another), can analyze movement behavior. In this paper, we study the problem of finding important trends in passenger movements between regions at different granularities. We define Origin (O), Destination (D), and Time (T) patterns (ODT patterns) and propose an algorithm that enumerates them. We propose optimizations that greatly reduce the search space and the computational cost of pattern enumeration. We also propose pattern variants (constrained patterns and top-k patterns) that could be useful to different application scenarios. We evaluate our methods on three real datasets and identify significant ODT flow patterns in them.

ACM Reference Format:
Chrysanthi Kosyfaki, Nikos Mamoulis, Reynold Cheng, and Ben Kao. 2018. Generalized Origin-Destination-Time Flow Patterns. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 12 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

In a transportation network, nodes represent stations (or districts) and edges model the connections between the nodes carrying information related to movement or flow (e.g., passengers travel between districts during different times of a day). Examples include car or taxi passengers moving from one district of a city to another, metro or railway passengers, global travelers by air, etc. Transportation companies and organizations collect large volumes of data from their passengers, regarding their origins, destinations, and times of their trips. Information on individual trips can be used in personalized services, after obtaining consent from the passengers. Other than that, it is hard to use such detailed data, mainly due to privacy constraints. However, aggregate information about passenger trips can be invaluable to the company since it can provide time-parameterized estimates and predictions about the passenger flow between regions, which can help to improve their service.

Problem We propose a novel analysis tool for transportation data, called *Origin-Destination-Time* (ODT) patterns. An ODT pattern captures information about the origins, destinations, and the volume (flow) of the passengers at different times of the day where the trips take place. We first use the application domain to define the finest granularity of regions on the map (e.g., each region corresponds to a metro station or a district) and also define the finest time intervals of interest (e.g., divide the 24-hour time interval of a day into 48 equal 30-minute timeslots or use short slots at peak hours and long ones at night). We call these *atomic regions* and *atomic timeslots*, respectively. This means that, a trip can be described by its origin region, its destination region, and the timeslot when the trip starts, i.e., as an (o, d, t) triple. For all (o, d, t) triples, where o and d are (different) atomic regions and t is an atomic timeslot, we measure the total number of passengers who took a trip from o to d at time t. The total flow of an (o, d, t) triple characterizes its importance; the triples with high flow are considered to be important and they are called *atomic ODT-patterns* (we drop the ODT prefix whenever the context is clear). In a *generalized* ODT-triple, denoted by (O, D, T), O and D are sets of *neighboring* atomic regions and T consists of one or more *consecutive* timeslots. An atomic (o, d, t) triple is a component of an (O, D, T) triple if $o \in O$, $d \in D$, and $t \in T$. (O, D, T) is *non-atomic*, if at least one of O, D, or T is non-atomic. Figure 1 exemplifies a generalized ODT pattern, which captures statistically significant movement in Brooklyn from boroughs {35,77,177} to boroughs {25,40,106} in the mornings of weekdays.

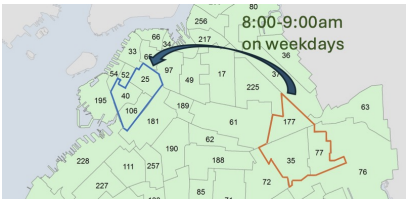

Figure 1: Example of an ODT pattern

The number of possible atomic region combinations that can form a generalized (i.e., non-atomic) region O or D is huge and it is not practical to consider all these combinations and their flows. At the same time, for a given generalized ODT triple, it is hard to estimate the flow quantity that can be deemed significant enough

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

to characterize the triple an interesting pattern. To solve these issues, we follow a *voting* approach, where we characterize an ODT triple as a pattern if at least a certain percentage of its constituent (o, d, t) triples are atomic patterns (i.e., they have large enough flow). This allows us to design and use a pattern enumeration algorithm, which, starting from the atomic patterns, identifies all ODT patterns progressively by synthesizing them from less generalized ODT patterns. We propose a number of optimizations, which significantly reduce the time spent for generating candidate patterns and measuring their significance.

Given the potentially large number of ODT patterns and the high cost of their enumeration, we also study practical ODT variants. First, we investigate the detection of patterns which are *constrained* to a subset of regions and timeslots, allowing us to focus on under-represented regions and to greatly reduce the pattern enumeration cost. Second, we study pattern detection by limiting the number of atomic regions and timeslots that a pattern may have. Finally, we define and solve the problem of finding the top-ranked patterns at each granularity level, by a novel algorithm that significantly outperforms the baseline approach of finding all patterns at each level and then selecting the top ones.

Applications The problem of identifying ODT patterns finds application in a wide variety of real-world transportation networks [45, 59, 67], such as road networks, railways, etc. For example, passenger movement patterns can facilitate the handling of emergencies or incidents by metro systems. Consider an accident that happened in Hong Kong subway system in December 2021[1]. Scheduled trips from some districts were cancelled and passengers had to be served by other means (i.e., buses). ODT patterns could help to schedule on-demand transportation for affected passengers as they could indicate origin-destination region pairs served by the disrupted route with heavy demand at the time of the day when the incident happened. In general, ODT patterns can help transportation companies to improve their schedule and in managing their available resources (e.g., fleet, employees). Another application is identifying the correlation between map districts for performing target-marketing, cross-district advertisements, or location planning. For example, if there is a heavy passenger flow between districts that are far from each other, the municipality may consider allocating new space for housing or opening new services near the residencies of passengers, to reduce traffic and improve the environmental conditions.

Novelty The input to our problem are not the entire trajectories of passengers, which is typically not available to transportation companies or organizations due to privacy constraints. Instead, we assume that we only have access to the (anonymized) trips of passengers, either individually or aggregated (e.g., the total number of passengers who took a trip from a specific origin to a specific destination at a specific time). For example, metro companies only have access to the entry and exit points of passengers, whereas public taxi trip data only include the origin/destination districts and time of the trip, but not the route. Hence, sequence or trajectory data mining methods are not applicable to our problem. In addition, our goal is also very different compared to the objective of sequence pattern mining. We study the identification of frequent generalized ODT triples, where the origins, destinations, and timeslots could be generalized to be a subgraph of neighboring regions or a time interval. On the other hand, the goal of sequence/trajectory mining methods is to find frequent subsequences. Table 1 summarizes representative works on trafflic flow mining (TFM), sequential and trajectory pattern mining (STM), origin-destination flow prediction (ODFP), and travel time estimation (TTE) problems and their differences compared to our work. In summary, all works consider time, but ours is the only work that (i) finds generalized ODT patterns, (ii) does not require in-between points of trajectories, and (iii) considers flow in pattern detection. For a detailed comparison to previous work, see Sec. 2.

[1]https://www.thestandard.com.hk/breaking-news/section/4/183861/(Video)-MTR-door-flew-off,-disrupting-peak-hour-service

Table 1: Previous works

Previous works	Characteristics			
	ODT Generalization	Time	Req. Traject.	Flow
TFM ([22, 25, 45, 63])	✗	✓	✗	✓
STM ([4, 5, 16, 24, 27, 29, 36])	✗	✓	✓	✗
ODFP ([10, 14, 23, 41, 53, 59])	✗	✓	✗	✓
TTE ([35, 44, 69, 70])	✗	✓	✓	✗
ODT patterns ([our work])	✓	✓	✗	✓

Outline Section 2 reviews related work on spatio-temporal pattern mining. In Section 3, we formally define the problem we study in this paper. Section 4 presents an algorithm for extracting spatio-temporal flow patterns and its optimizations. In Section 5, we define interesting variants of flow patterns and propose algorithms for their enumeration. Section 6 evaluates our methods on real networks with different characteristics. Finally, Section 7 concludes the paper with a discussion about future work.

2 RELATED WORK

Mining spatiotemporal patterns is well-studied in the literature [6, 8, 9, 11–15, 18, 18, 20, 21, 25, 26, 30, 32, 33, 39, 40, 42, 43, 46–48, 50–52, 55, 57, 58, 60–62, 65, 68, 72–75]. Given a graph G, the main objective is to count/enumerate/identify spatial events, correlations, or sequences that repeat themselves over time. We are the first to address the generalization of spatial regions considering their adjacency to capture the passengers movement more effectively.

Sequential and Trajectory Pattern Mining Sequential pattern mining methods [17, 26] discover interesting subsequences, based on occurence frequency. Agrawal and Srikant [4] introduced the concept of sequential item-purchase patterns over a database of time-ordered customer transactions. Similar to sequence mining, trajectory pattern mining is the problem of identifying frequent subsequences in space and time [16, 29, 36, 37, 56]. Spatio-temporal patterns are discovered from raw GPS data, which capture routes or passenger movements. Giannotti et al. [27] suggested the discovery of regions of interest in trajectory pattern mining, as considering sequences of predefined regions reduces the complexity of the problem. In the same spirit, Cao et al. [17] automatically segment the space into spatial regions by clustering the tracked object locations; then, they identify sequential patterns of regions that repeat themselves over time. Choi et al., [19] introduce a tool for discovering all regional movement patterns in semantic trajectories. In a semantic trajectory [5], each point of a trajectory not only represents a specific location but also contains a semantic category. Patterns can

then be defined based on semantics and locations over time. Fan et al. [24] propose scalable trajectory mining methods using Spark.

Our problem is very different compared to previous work on trajectory, sequence, and graph mining. First, we are not interested in finding frequent paths (subsequences, subgraphs), but in finding hot combinations of trip origins, destinations, and timeslots. Second, we do not search for patterns at the finest granularity only, but looking for patterns where any of the three ODT components are generalized at an arbitrary granularity. Furthermore, we only use a *weak monotonicity* property to construct and validate generalized patterns from more detailed ones, which means that the classic Apriori algorithm (and its variants) [1–3, 31, 38] cannot be readily applied to solve our problem.

Traffic Flow Mining Mining traffic flow patterns [22, 25, 45, 63] is another related problem to ODT patterns. Liu et al. [45] studied the problem of extracting traffic flow knowledge from transportation data, i.e., pairs of POIs on the map that have significant traffic flow between them. Although we also measure flow between regions, none of these previous works studies flow patterns at *different granularities*, where regions and time periods may consist of multiple atomic elements; hence, previous works cannot be applied to solve our problem. Moreover, they require in-between locations, which may not be available and are not relevant to our problem's objective.

Travel Time Estimation The main objective of *Travel Time Estimation (TTE)* is to estimate the time to travel from a specific origin location on the map to a specific destination at a given time [35, 69, 70]. Lin et al., [44] propose an indexing approach (oracle) for estimating the travel time between different regions at different times of the day. Although they also use the term *ODT*, our goals are completely different. While our main objective is to identify (generalized) Origin-Destination-Time patterns considering the (passenger) flow, their goal is to predict the travel time from a specific detailed region O to a detailed destination D at a time T.

Origin-Destination Flow Prediction Predicting the origin-destination traffic flow has been well studied transportation research [10, 12, 14, 23, 28, 41, 49, 53, 54, 59, 64, 66, 71]. The main objective is to estimate the flow (e.g., number of passengers in a transportation network or cars in a road network) between origin-destination pairs. A typical approach is to construct a Origin-Destination matrix [7], where each cell represents the trips between an origin and a destination. Tu et al., [57] study the stability of travel patterns on different times of the day and use predefined geographic units to group the origin-destination pairs of each journey. Behara et al., [14] study the problem of OD flow prediction by proposing a geographical window-based structural similarity (GSSI) index that captures statistics of geographically correlated OD pairs. OD pairs correlate if they share similar structural properties and therefore expand to neighbor regions. Their proposed solution can help identifying local patterns more efficiently. Similarly, [12] proposes a DBSCAN framework to identify travel patterns in multi-density high dimensional matrices. Although [12, 14] share similarities with our work, the set up of the problem is different. First, they use similarity metrics to group regions together when defining OD pairs. In our case, we perform region and time generalization based on (geographic) adjacency and connectivity in a neighborhood graph. Wang et al., [59] develop a model for predicting the flow density in different origin-destination pairs. To this end, they represent a city as a grid and take into consideration regions that may have a significant amount of flow compared to other less important regions. The goal of this work is similar to ours, however they use additional data such as POIs and area proximity, which, in our case, are not available. In summary, our framework can identify flow patterns that can be used to predict demands in the flow between generalized regions and at (generalized) timeslots efficiently using the minimum available information.

3 DEFINITIONS

In this section, we formally define ODT patterns and the graph wherein they are identified. Table 2 shows the notations used frequently in the paper.

Table 2: Table of notations

Notations	Description
$G(V, E)$	region neighborhood graph
r_i	atomic region
R_i	region
t_i	atomic timeslot
T_i	timeslot
P	atomic ODT pattern or triple
$P.O$	pattern/triple origin
D	pattern/triple destination
T	pattern/triple timeslot
$\sigma(P)$	support of atomic ODT pattern P
s_a	support of atomic regions and timeslots
s_r	ratio threshold
P.cnt	number of atomic patterns in ODT pattern P
$\mathcal{P}_\ell/\mathcal{T}_\ell$	Set of ODT patterns/triples at level ℓ

The main input to our problem is a *trips* table, which records information about trips from origins to destinations at different times. Each origin/destination is a minimal region of interest on a map (e.g., a district, a metro station, etc.), called *atomic region*. Atomic regions are specified by the application; they could be pre-defined districts or stations/stops and their nearby area. Spatial clustering can also be applied on precise locations (e.g., taxi pickup/dropoff locations) to define a set of atomic regions, as in [17]. Let V be the set of all atomic regions. An undirected graph $G(V, E)$ defines the neighboring relations between atomic regions; there is an edge (v, u) in E iff $v \in V$ and $u \in V$ are neighbors on the map. Finally, the timeline is divided into periods that repeat themselves (e.g., 24-hours each) and each period is discretized into time ranges (e.g., 48 30-minute slots). Discretization of time can be uniform or it may follow the data distribution (e.g., using the boundaries of an equi-depth histogram). Each minimal time range of interest is called *atomic* timeslot. Periods can also be classified (e.g., weekdays vs. weekends); hence atomic timeslots may refer to different period classes (e.g., 8:00-8:30 on weekdays).

Definition 3.1 (Region/Timeslot). A region r is a subset V' of V, such that the induced subgraph $G'(V', E')$ of G is connected. A timeslot T is a continuous sequence of atomic timeslots.

Definition 3.2 (Generalization of a region/timeslot). A region R_1 is a generalization of region R_2 iff $R_2 \subset R_1$. A timeslot T_1 is a generalization of timeslot T_2 iff $T_2 \subset T_1$.

Definition 3.3 (Minimal generalization of an region/timeslot). A region R_1 is a minimal generalization of region R_2 iff $R_2 \subset R_1$ and $R_1 - R_2$ is an atomic region. A timeslot T_1 is a minimal generalization of timeslot T_2 iff $T_2 \subset T_1$ and $T_1 - T_2$ is an atomic timeslot.

For example, region $\{B, C, D\}$ is a minimal generalization of $\{B, D\}$. Symmetrically, $\{B, D\}$ is a minimal specialization of $\{B, C, D\}$.

We found that the start and end times of individual trips between the same origin and destination are strongly correlated. Hence, we can map each trip in the trips table to an *atomic ODT triple* (o, d, t), where o is the origin region of the trip (GPS locations can be mapped to the nearest spatially $v \in V$), d is the trip's destination region, and t is the atomic timeslot that contains the trip's origin time.[2]

Definition 3.4 (Atomic ODT triple). A triple (o, d, t) is atomic if:

- o is an atomic region
- d is an atomic region
- t is an atomic timeslot
- $o \neq d$

Given an atomic ODT triple P, the *support* $\sigma(P)$ of P is the total number of passengers (flow) of the trips that are mapped to P. For example, the top-left of Figure 2 shows a map and an individual trip in it. The top-right of Figure 2 has the *aggregated trips table*, which contains all atomic (o, d, t) triples, after aggregating all trips that correspond to the same (o, d, t). For instance, trips (B, D, 9:20, 2) and (B, D, 9:29, 1) are merged to triple $(B, D, 18)$ with total flow 3.[3] Next, we define generalized (i.e., non-atomic) ODT triples.

Definition 3.5 (ODT triple). An ODT triple (O, D, T) consists of a region O, a region D, and a timeslot T, such that $O \cap D = \emptyset$.

Definition 3.6 (ODT triple generalization). An ODT triple P_1 is a generalization of ODT triple P_2 if for each $X \in \{O, D, T\}$, either $P_1.X = P_2.X$ or $P_2.X \subset P_1.X$, and for at least one $X \in \{O, D, T\}$, $P_2.X \subset P_1.X$.

Definition 3.7 (Minimal generalization of ODT triple). An ODT triple P_1 is a minimal generalization of ODT triple P_2 if one of the following holds:

- $P_1.O = P_2.O$, $P_1.D = P_2.D$ and $P_1.T$ is a minimal generalization of $P_2.T$
- $P_1.D = P_2.D$, $P_1.T = P_2.T$ and $P_1.O$ is a minimal generalization of $P_2.O$
- $P_1.O = P_2.O$, $P_1.T = P_2.T$ and $P_1.D$ is a minimal generalization of $P_2.D$

We now turn to the definition of ODT patterns; we start by atomic ODT patterns and then move to the generalized ODT patterns.

Definition 3.8 (Atomic ODT pattern). Let AT_r be the set of atomic ODT triples with non-zero support. Given a threshold s_a, $0 < s_a \leq 1$, an atomic ODT triple P is called an atomic ODT pattern if $\sigma(P)$ is in the top $s_a \times |AT_r|$ supports of triples in AT_r.

[2]Our definitions and techniques can be extended for data inputs where there is no correlations between the begin and end times of trips; in this case, we should map trips to (o, d, st, et) quadruples and patterns should be ODSE quadruples.

[3]All time moments between 9:00 and 9:30 are generalized to timeslot 18, which is the 18th slot in 30-minute intervals, starting from 00:00-00:30 (mapped to 0).

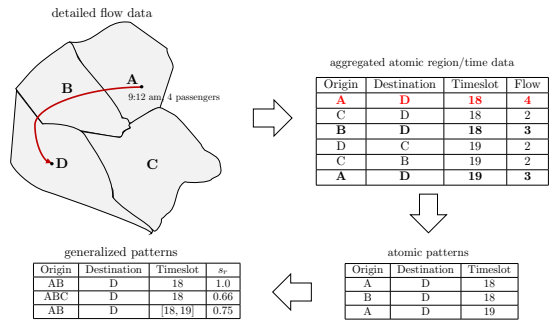

Figure 2: A detailed example

The rationale of using s_a is that we want to restrict the set of important atomic ODT triples, by considering as patterns the triples that have large total flow (accumulated over the repeated instances of the triples in the trips table). Figure 2 (bottom-right) shows the atomic ODT patterns for our running example if $s_a = 0.5$.

Definition 3.9 (ODT pattern). An ODT pattern P is an ODT triple where:

- the ratio of atomic triples in P, which are atomic patterns, is at least equal to a *minimum ratio* threshold s_r
- there exists a minimal specialization of P, which is an ODT pattern

The number of atomic triples in P, which are atomic patterns is denoted by P.cnt. In the example of Figure 2, if $s_r = 0.6$, $(AB, D, 18)$ is a (generalized) ODT pattern where origins A and B have significant joint flow to destination D at timeslot $t = 18$, because the pattern includes two out of three atomic patterns. The second condition of Definition 3.9 is a *sanity* constraint, which prevents a generalized triple P from being characterized as a pattern if there is no minimal specialization of P that is also a pattern; intuitively, a pattern should have at least one minimal specialization which is also a pattern (weak monotonicity).

A pattern (triple) P is said to be level-ℓ pattern (triple) if the total number of atomic elements in it (regions and timeslots) is ℓ. Hence, atomic patterns are level-3 patterns, since they contain exactly 3 elements (i.e., two atomic regions and one atomic timeslot). Similarly, triple $(A, BC, [1, 3])$ is a level-6 triple because it includes 1 atomic region in its origin, 2 atomic regions in its destination, and 3 atomic timeslots in its time-range (note that $[1, 3]$ includes atomic timeslots $\{1, 2, 3\}$).

4 PATTERN EXTRACTION

To find the ODT patterns, we first start by finding the atomic ODT patterns, i.e., the (o, d, t) triples which are frequent/significant, where o and d are atomic regions and t is an atomic timeslot. This is trivial and can be done by one pass over the aggregated trips data, where the occurrence of each (o, d, t) triple is unique and by selecting the top s_a ratio of them as atomic patterns. Our most challenging task is then to define an algorithm that progressively synthesizes non-atomic patterns from atomic patterns and prunes the search space effectively.

Recall that a non-atomic triple $P = (O, D, T)$ is a pattern if at least a ratio $s_r > 0$ of its included atomic triples are patterns. Hence,

by definition, a non-atomic pattern generalizes at least one atomic pattern (o, d, t). The pattern synthesis algorithm uses the set of atomic patterns and the region neighborhood graph G to form the non-atomic patterns. Given an existing (O, D, T) pattern P, we attempt a minimal generalization of P by including into the set O a neighboring atomic region to the existing regions in O, or doing the same for set D, or adding an atomic neighboring timeslot to T.

The challenge is to prune candidate generalizations that cannot be patterns. For this, we need a fast way to compute (or bound) the number of contributing (newly added to P) atomic patterns to the ratio of the candidate.

4.1 Baseline Algorithm

We now present a baseline algorithm for enumerating all the atomic and extended ODT patterns in an input graph $G(V, E)$. The first step of Algorithm 1 is to scan all trips data and compute the support counts of all atomic triples $\mathcal{T}_3$. Then, it finds the set $\mathcal{P}_3$ of atomic patterns, i.e., the triples having support count at least equal to *minsup*, which is the support count of the $s_a \cdot |\mathcal{T}_3|$-th triple in $\mathcal{T}_3$ in decreasing support order. All triples (patterns) in $\mathcal{T}_3$ ($\mathcal{P}_3$) have exactly three atomic elements (regions or timeslots). The algorithm progressively finds the patterns with more atomic elements. Recall that a triple (pattern) P is at level ℓ, i.e., in set $\mathcal{T}_\ell$ ($\mathcal{P}_\ell$) if it has ℓ atomic elements; we also call P a level-ℓ triple (pattern). Candidate patterns *CandP* at level $\ell + 1$ are generated by either adding an atomic region at O or an atomic region at D or an atomic timeslot at T, provided that the resulting triple is valid according to Definition 3.5. If a *CandP* has been considered before, it is disregarded. This may happen because the same triple can be generated from two or more different triples at level ℓ. For example, candidate pattern $(AB, C, 1)$ could be generated by pattern $(A, C, 1)$ (by extending region A to region AB) and by $(B, C, 1)$ (by extending region B to region AB). Hence, we keep track at each level ℓ the set of triples that have been considered before, in order to avoid counting the same candidate twice.[4]

To check whether a candidate *CandP* not considered before is a pattern, we need to divide the number *CandP*.cnt of atomic patterns included in *CandP* by the total number *CandP*.card of atomic triples in *CandP*. If this ratio is at least s_r, then *CandP* is a pattern. *CandP*.card can be computed algebraically: it is the product of atomic number elements in each of the three ODT components. For example, $(AB, CD, [1, 3])$.card $= 2 \cdot 2 \cdot 3 = 12$ because there are 12 atomic triples in $(AB, CD, [1, 3])$, i.e., combinations of elements $\{A, B\}$, $\{C, D\}$, and $\{1, 2, 3\}$. To compute *CandP*.cnt fast, we can take advantage of the fact that we already have P.cnt, i.e., the number of atomic patterns in the generator pattern. We only have to compute the P'.cnt for the *difference* $P' = CandP - P$ between *CandP* and P, which is the triple consisting of the extension element in the extended dimension (one of O, D, T) together with the element-sets in the intact dimensions (two of O, D, T). For example, if $P = (A, CD, [1, 2])$ and $CandP = (AB, CD, [1, 2])$, then $P' = (B, CD, [1, 2])$. To compute P'.cnt, Algorithm 1 enumerates all atomic triples in P' to check whether they are patterns. It then sums up P.cnt and P'.cnt to derive *CandP*.cnt.

[4]Since a pattern at level $\ell+1$ requires at least one and not all its minimal specializations to be patterns, an id-numbering scheme for atomic regions, which would extend patterns by only adding elements that have larger id would not work. For example, if both $(A, C, 1)$ and $(B, C, 1)$ are patterns, $(AB, C, 1)$ can be generated by both of them; however, if just $(B, C, 1)$ is a pattern, $(AB, C, 1)$ can only be generated by $(B, C, 1)$.

Algorithm 1 Baseline Algorithm for finding all ODT patterns

Require: a region graph $G(V, E)$; a trips table; a minimum support s_a for atomic ODT patterns; a minimum support ratio s_r for non-atomic ODT patterns

```
1:  T3 = atomic triples computed from trips table
2:  P3 = triples in T3 with support ≥ sa × |T3|
3:  for all atomic triples P ∈ T3 do
4:      P.cnt = 1 if P ∈ P3, else P.cnt=0
5:  end for
6:  ℓ = 3
7:  while |Pℓ| > 0 do                          ▷ Pℓ = set of level-ℓ patterns
8:      Pℓ+1 = ∅                               ▷ Initialize pattern set at level ℓ + 1
9:      for each P in Pℓ do
10:         for each minimal generalization CandP of P do
11:             if CandP not considered before then
12:                 P′ = CandP − P
13:                 CandP.cnt = P.cnt + P′.cnt
14:                 if CandP.cnt / CandP.card ≥ sr then
15:                     add CandP to Pℓ+1
16:                 end if
17:             end if
18:         end for
19:     end for
20:     ℓ = ℓ + 1
21: end while
```

Figure 3 exemplifies the pattern enumeration process in our running example (see Figure 2). Atomic pattern $P = (A, D, 18)$ can be generalized by adding to the origin any of the neighbors of atomic region A, to the destination any of the neighbors of atomic region D, and to timeslot 18 either timeslot 17 or timeslot 19. Each of these generalization forms a candidate pattern *CandP* at level 4. Counting the support of these candidates requires counting only the difference P'. For example, to count the support of $(AB, D, 18)$, we only have to add to the support of $P = (A, D, 18)$ the support of $P' = (B, D, 18)$, which is 1. Then, the support of $(AB, D, 18)$ is found to be 2. Assuming that $s_r = 0.6$, $CandP = (AB, D, 18)$ is a pattern, since the ratio of atomic patterns in it is $1.0 \geq s_r$. All patterns that stem from $P = (A, D, 18)$ up to level 5 are emphasized in Figure 3; these are used to generate candidate patterns at the next levels.

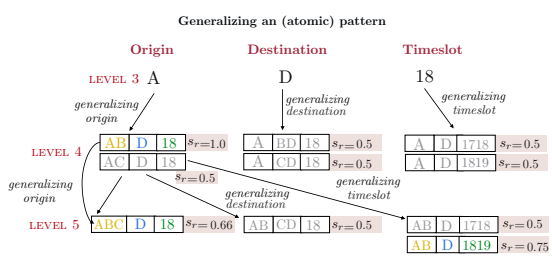

Figure 3: Pattern enumeration example

Complexity Analysis In the worst case, all valid combinations of regions and timeslots can be considered as candidate patterns. In other words, each subset of V can be the union of O and D and for each such subset of size k can be split in $2^k - 2$ ways. Hence, the number of possible OD pairs is $\sum_{k=1}^{|V|-1} \binom{|V|}{k}(2^k - 2)$. If S is the number of atomic timeslots, then the number of possible (generalized) timeslots to be included in a candidate pattern is $2^{|S|}$. Hence, the worst-case space/time complexity of ODT pattern enumeration is $O\left(2^{|S|} \sum_{k=1}^{|V|-1} \binom{|V|}{k}(2^k - 2)\right)$.

4.2 Optimizations

We now discuss some optimizations to the baseline algorithm, which can greatly enhance its performance in practice.

Avoid re-counting P'. The first approach is based on *caching* the ODT triples that have been counted before. Instead of computing P'.cnt directly for $P' = CandP - P$, we first check whether P'.cnt is already available. This requires us to cache the counted triples and their supports at each level in a hash table. Hence, before counting P', we first search the hash table, which caches the triples at level $|P'|$ to see if P' is in there. In this case, we simply use P'.cnt instead of computing it again from scratch.

Fast check for zero support of P'. The second optimization is based on the observation that for some pairs (o, d) of atomic regions, there does not exist any timeslot t, such that (o, d, t) is an atomic pattern in $\mathcal{P}_3$. For example, if o and d are remote regions on the map, it is unlikely that there is significant passenger flow that connects them at any time of the day. We take advantage of this to avoid counting any P' which may not include atomic patterns. Specifically, for each atomic region r, we record (i) $r.dests$, the set of atomic regions r', such that there exists a (r, r', t) pattern in $\mathcal{P}_3$; and (ii) $r.srcs$, the set of atomic regions r', such that there exists a (r', r, t) pattern in $\mathcal{P}_3$. If, in Algorithm 1, $CandP$ is a minimal generalization of P, by expanding $P.O$ to include a new atomic region r, then $P' = CandP - P$ should only include r in $P'.O$. If $P'.D \cap r.dests = \emptyset$, then P' does not include any atomic patterns and $CandP$.cnt is guaranteed to be equal to P.cnt. Hence, we can skip support computations for P'. Symmetrically, if $CandP$ is a minimal generalization of P, by expanding $P.D$ to include a new atomic region r, then $P' = CandP - P$ should only include r in $P'.D$. If $P'.O \cap r.srcs = \emptyset$, then $CandP$.cnt is guaranteed to be equal to P.cnt. Overall, by keeping track of $r.dests$ and $r.srcs$ for each atomic region r, we can save computations when counting the supports of patterns. Since the space required to store $r.dests$ and $r.srcs$ is $O(|V|)$ in the worst case, the total space complexity of these sets is $O(|V|^2)$. This cost is bearable, because our problem typically applies on transportation networks or district neighborhood graphs in urban maps, where the number of vertices in V is rarely large.

Improved neighborhood computation. The minimal generalizations of a pattern P are generated by minimally generalizing $P.O$, $P.D$, and $P.T$. The generalization of $P.T$ is trivial as we add one atomic timeslot before the smallest one in $P.T$ or after the largest one in $P.T$. On the other hand, computing the minimal generalizations of a region R (i.e., $P.O$ or $P.D$) can be costly if done in a brute-force way. The naive algorithm tries to add to $P.O$ all possible neighbors of each atomic region $r \in R$ and for each such neighbor not in $P.O$ and $P.D$ it measures the support of the corresponding generalized pattern P', if P' was not considered before. Since the same P' can be generated by multiple P, checking whether P' has been considered before can be performed a very large number of times with a negative effect in the runtime. We design a neighborhood computation technique for a region R, which avoids generating the same P' multiple times. The main idea is to collect first all neighbors of all $r \in R$ in a set N and then compute (in one step) $N - P.O - P.D$, i.e., the set of regions r that minimally expand R to form the minimal generalizations of P.

Indexing atomic patterns. As another optimization, we employ a prefix-sum index which can help us to compute an upper bound of the support of P'. The main idea comes from indexes used to compute range-sums in OLAP [34]. Let N be the number of atomic regions and M be the number of atomic timeslots. Consider a $N \times N \times M$ array A, where each cell corresponds to an atomic ODT triple. The cell includes a 1 if the corresponding atomic ODT triple is a pattern; otherwise the cell includes a 0. In addition, consider a 3D array R with shape $(N+1) \times (N+1) \times (M+1)$. Each element $R[i][j][k]$ of R is the sum of all elements $A[i'][j'][k']$ of A, such that $i' \leq i$, $j' \leq j$, and $k' \leq k$ $R[i][j][k] = 0$ if any of i, j, k is 0. R is the *prefix-sum* array of A, as illustrated in Figure 4.

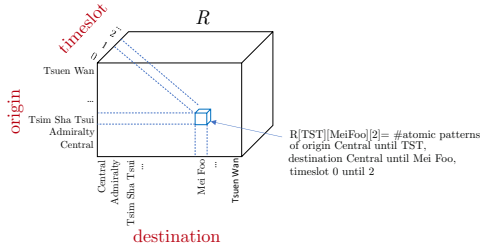

Figure 4: Prefix sum example

Consider a 3D range $[a, b], [c, d], [e, f]$, where $0 < a \leq b \leq N$, $0 < c \leq d \leq N$, and $0 < e \leq f \leq M$. The sum of values in A inside this range can be computed as follows:

$$
\begin{aligned}
&R[b][d][f] \\
&- R[a-1][d][f] - R[b][c-1][f] - R[b][d][e-1] \\
&+ R[a-1][c-1][f] + R[b][c-1][e-1] + R[a-1][d][e-1] \\
&- R[a-1][c-1][e-1]
\end{aligned}
$$

Now, consider a P' which needs to be counted. P' includes a set of atomic origin regions, a set of atomic destination regions, and a set of atomic timeslots. The atomic timeslots are guaranteed to be a continuous sublist of regions in the corresponding dimension of the 3D array A, starting, say, from timeslot e and including up to timeslot f. However, the region sets in P' are not guaranteed to be continuous. Still, the 3D range $[a, b], [c, d], [e, f]$, where a (c) is the origin (destination) region in P' with the smallest ID and b (d) is the origin (destination) region in P' with the largest ID is guaranteed to be a superset of atomic triples in P'. Hence, the prefix-sum index can give us in $O(1)$ time an *upper bound* of the number of atomic patterns in P'. If this upper bound is added to the support of P and the resulting support is less than s_r, then $CandP$ is definitely not a pattern, so we can avoid counting P'. To maximize the effectiveness of this optimization, we give IDs to regions by applying a depth-first traversal of G from the vertex with the smallest degree.

5 PATTERN VARIANTS

In this section, we explore alternative problem definitions and the corresponding ODT pattern extraction algorithms that can be more useful than our general definition in certain problem instances. In particular, we observe that the number of patterns can be huge even if relatively small s_a and large s_r are used. In addition, setting global thresholds may not be "fair" for some regions which are under-represented in the data. To address these issues, we propose

(i) size-bounded patterns, (ii) constrained-pattern search, and (iii) rank-based patterns.

5.1 Size-bounded Patterns

The first type of constraint that we can put to limit the number of patterns is on the sizes of the pattern O,D, and T components. Specifically, we can set an upper bound B_O to the number of atomic regions in an origin region of a pattern. Similarly, we can limit the number of regions in D to at most B_D and the number of atomic timeslots to at most B_T. In effect, this limits the number of levels that we use for pattern search to $B_O \cdot B_D \cdot B_T$. For pattern enumeration, we use the same algorithms and optimizations discussed in Section 4, but with the constraints applied whenever we expand a pattern to generate the candidate patterns at the next level.

5.2 Constrained Patterns

Another way to control the number of the patterns, but also focus on specific regions and/or timeslots that are under-represented in the entire population is to limit the domain of atomic regions and timeslots. Specifically, we give as parameter to the problem the set of atomic regions $V_O \subseteq V$ that we are interested in to act as origins the set $V_D \subseteq V$ of regions that can act as destinations, and $T_R \subseteq T$, a restricted contiguous subsequence of the entire sequence of atomic timeslots T to be used as timeslots in the patterns. The induced subgraphs by V_O and V_D should be connected, in order to potentially have the entire V_O (and/or V_D) as an origin (destination) of a pattern. For example, if a data analyst is interested in flow patterns from South Manhattan to Queens in afternoon hours, she could include in V_O (resp. V_D) all the atomic regions in South Manhattan (resp. Queens) and restrict the timeslots to be used in patterns to only afternoon hours. Depending on the sizes of V_O, V_D, and T_R pattern enumeration can be significantly faster compared to unconstrained pattern search.

5.3 Rank-based patterns

Another way to control the number of patterns and still not miss the most important ones is to regard as patterns, at each level, the k triples with the highest support and prune the rest of them as non-patterns. This is achieved by replacing the minimum ratio threshold s_r by a parameter k, which models the ratio of eligible triples at each level which are considered to be important. More formally, let $\mathcal{T}_\ell$ be the set of triples at level ℓ, which are minimal generalization of patterns at level $\ell - 1$. The patterns $\mathcal{P}_\ell$ at level ℓ are the k triples in T_ℓ having the largest number of atomic patterns.

Definition 5.1 (ODT pattern (rank-based)). An ODT triple P at level ℓ is a rank-based ODT pattern if:

- there exists a minimal specialization of P which is a rank-based ODT pattern
- there are no more than k minimal generalizations of level-$(\ell - 1)$ rank-based ODT patterns that include more frequent atomic patterns than P.

5.3.1 Baseline approach for rank-based pattern enumeration. A baseline approach for enumerating rank-based patterns is to generate all eligible triples at each level ℓ, which are minimal generalizations of patterns at level $\ell - 1$. For each such triple, count its support (i.e., number of atomic patterns included in it). We may use the optimizations proposed in Section 4.2, to reduce the cost of generating ODT triples that are candidate patterns and counting their supports. After generating all triples and counting their supports, we select the top-k ones as patterns. Only these patterns are used to generate the candidate patterns at level $\ell + 1$.

5.3.2 Optimized rank-based pattern enumeration. To minimize the number of generated triples at each level ℓ and the effort for counting them, we examine the patterns at $\ell - 1$ in decreasing order of their potential to generate triples that will end up in the top-k triples at level ℓ. Hence, we access the patterns P at level $\ell - 1$ in decreasing order of their support P.cnt. For each such pattern P and for each minimal generalization $CandP$ of P, we first compute the potential of $P' = CandP - P$ to add to the support $CandP$.cnt (initially $CandP$.cnt = P.cnt). If, by adding the maximum possible P'.cnt to $CandP$.cnt, $CandP$.cnt cannot make it to the top-k ℓ-triples so far, then we prune $CandP$ and avoid its counting. The maximum possible P'.cnt can be computed based on the following lemma:

Lemma 5.2. *The maximum possible P'.cnt that can be added to P.cnt, to derive the support of $CandP$ is as follows:*

- *If $CandP$ is generated by minimally generalizing $P.O$, then P'.cnt equals $|P.D| \cdot |P.T|$.*
- *If $CandP$ is generated by minimally generalizing $P.D$, then P'.cnt equals $|P.O| \cdot |P.T|$.*
- *If $CandP$ is generated by minimally generalizing $P.T$, then P'.cnt equals $|P.O| \cdot |P.D|$.*

Let θ be the k-th largest support of the triples generated so far at level ℓ. If for the next examined P from level $\ell - 1$ to generalize, P cannot be generalized to a $CandP$ that may end up in the top-k level-ℓ triples, then we can immediately prune P. The condition for pruning P follows:

Lemma 5.3. *If $P.cnt + \max\{|P.D| \cdot |P.T|, |P.O| \cdot |P.O|, |P.O| \cdot |P.D|\} \leq \theta$, then no minimal generalization of P can enter the set of top-k level-ℓ ODT triples.*

Based on the above lemmas, we can prove the correctness of our enumeration algorithm for rank-based ODT patterns, described by Algorithm 2. The algorithm computes first all level-3 patterns $\mathcal{P}_3$, based on the atomic pattern support threshold s_a (Lines 3–5). Having the patterns at level ℓ, the algorithm organizes those at level $\ell + 1$ in a priority queue (minheap) $\mathcal{P}_{\ell+1}$ of maximum size k. We consider all patterns P at level ℓ in decreasing order of support P.cnt, to maximize the potential of generating level-$(\ell + 1)$ triples of high support early. For each such pattern P, we first check if P can generate any level-$(\ell + 1)$ triple that can enter the set $\mathcal{P}_{\ell+1}$ of top-k triples so far at level $\ell + 1$, based on Lemma 5.3. If this is not possible, then P is pruned. Otherwise, we attempt to generalize P, first by adding an atomic region to $P.O$. If the maximum addition to P.cnt by such an extension cannot result in a $CandP$ that can enter the top-k at level $\ell + 1$ (based on Lemma 5.2), then we do not attempt such extensions; otherwise we try all such extensions and measure their supports (Lines 15 to 25). We repeat the same for the possible extensions of $P.D$ and $P.T$. After $\mathcal{P}_{\ell+1}$ has been finalized, we use it to generate the top-k patterns at the next level. Since the number of levels for which we can generate patterns can be very

Algorithm 2 Optimized Algorithm for enumerating rank-based ODT patterns

Require: a region graph $G(V, E)$; a trips table; a minimum support s_a for atomic ODT patterns; number k of top patterns to be generated at each level; maximum level considered ($maxl$)

1: $\mathcal{T}_3$ = atomic triples computed from trips table
2: $\mathcal{P}_3$ = triples in $\mathcal{T}_3$ with support $\geq s_a$
3: **for** all atomic triples $P \in \mathcal{T}_3$ **do**
4: P.cnt = 1 if $P \in \mathcal{P}_3$, else P.cnt=0
5: **end for**
6: $\ell = 3$
7: **while** $|\mathcal{P}_\ell| > 0$ and $\ell < maxl$ **do** ▷ extend level-ℓ patterns
8: $\mathcal{P}_{\ell+1} = \emptyset$ ▷ Initialize k-minheap with level-$(\ell+1)$ patterns
9: **for** each P in $\mathcal{P}_\ell$ in decreasing order of P.cnt **do**
10: **if** $|\mathcal{P}_{\ell+1}| = k$ and P.cnt+max$\{|P.D| \cdot |P.T|, |P.O| \cdot |P.O|, |P.O| \cdot |P.D|\} \leq \mathcal{P}_{\ell+1}$.top.cnt **then**
11: **continue** ▷ Prune P based on Lemma 5.3
12: **end if**
13: **if** $|\mathcal{P}_{\ell+1}| = k$ and P.cnt+ $|P.D| \cdot |P.T| \leq \mathcal{P}_{\ell+1}$.top.cnt **then**
14: **for** each minimal generalization $CandP$ of P by origin **do**
15: **if** $CandP$ not considered before **then**
16: $P' = CandP - P$
17: $CandP$.cnt = P.cnt + P'.cnt
18: **if** $|\mathcal{P}_{\ell+1}| < k$ **then**
19: add $CandP$ to $\mathcal{P}_{\ell+1}$
20: **else**
21: **if** $CandP$.cnt $> \mathcal{P}_{\ell+1}$.top.cnt **then**
22: update $\mathcal{P}_{\ell+1}$ with $CandP$
23: **end if**
24: **end if**
25: **end if**
26: **end for**
27: **end if**
28: **if** $|\mathcal{P}_{\ell+1}| = k$ and P.cnt+ $|P.O| \cdot |P.T| \leq \mathcal{P}_{\ell+1}$.top.cnt **then**
29: **for** each minimal generalization $CandP$ of P by dest. **do**
30: Lines 15 to 25 above
31: **end for**
32: **end if**
33: **if** $|\mathcal{P}_{\ell+1}| = k$ and P.cnt+ $|P.O| \cdot |P.D| \leq \mathcal{P}_{\ell+1}$.top.cnt **then**
34: **for** each minimal generalization $CandP$ of P by time **do**
35: Lines 15 to 25 above
36: **end for**
37: **end if**
38: **end for**
39: $\ell = \ell + 1$
40: **end while**

large, Algorithm 2 takes as a parameter the maximum level $maxl$ for which we are interested in generating patterns.

6 EXPERIMENTS

In this section, we evaluate the performance of our proposed algorithms on real datasets. All methods were implemented in Python3 and the experiments were run on a Mac Mini with a M2 processor and 16GB memory. The source code is publicly available.[5]

6.1 Datasets

For our experiments, we used three real datasets; NYC taxi trips, MTR network trips and Flights. Below, we provide a detailed description for each of them.

NYC taxi trips: We processed 7.5M trips of yellow taxis in NYC in January 2019, downloaded from TLC[6]. Each record represents a taxi trip and includes the pick-up and drop-off taxi zones (different regions in NYC), the date/time of the pick-up, and the number of passengers who took the trip. We converted all time moments to 48 time-of-day slots (one slot per 30min intervals in the 24h). Then, we aggregated the data by merging all trips having the same origin, destination, and timeslot, and summing up the total number of passengers in all these trips to a total passenger flow, as explained in Section 3. This way, we ended up having 373460 unique ODT combinations (atomic ODT triples), which we used as input to our pattern enumeration algorithms. In addition, we used the maps posted at the same website to construct the neighboring graph G between the atomic regions (taxi zones). In G, we connected all pairs of atomic regions that share boundary points or are separated by water boundaries.

MTR trips: The Mass Transit Railway (MTR)[7] is the biggest and the most popular public transport network operating in Hong Kong. The system consists of 168 stations, serving the areas of Hong Kong island, Kowloon and New Territories. We consider each station as an atomic region; we created the neighborhood graph G for them by linking stations that are next to each other in the network. MTR Corp. provided us with aggregate data for all passenger trips taken in September 2019. Specifically, for each atomic ODT triple, where the origin and destination are MTR stations and T is one of the 48 atomic timeslots, we have the total number of passenger trips in Sep. 2019. The total number of atomic ODT triples is 253497.

Flights: We extracted information for 5.8M US flights in 2015 from Kaggle[8]. In this dataset, we consider as atomic regions 319 airports in North America that appear in the file. Since the number of passengers in each flight was not given in the original data, we randomly generated a number between 50 and 200. We followed the same procedure as in for the two previous datasets; namely, we converted the original flights data into a table with atomic ODT triples. The total number of resulting ODT triples is 17623. To create the neighbor graph G, we follow the same logic as the two previous datasets; we connect atomic regions in neighboring states.

6.2 Pattern enumeration

We start by evaluating the performance of our pattern enumeration algorithm (Section 4.1) and its optimizations (Section 4.2). Specifically, we compare the following methods:

- Baseline: Algorithm 1.
- AV: Algorithm 1 with avoid recounting P'.
- AVFC: Algorithm 1 with avoid recounting P' and fast check for zero support of P'.
- AVFCIN: Algorithm 1 with avoid recounting P', fast check, and improved neighborhood optimizations.
- OPT: Algorithm 1 with all four optimizations.

Figure 5 shows the costs and number of enumerated patterns of all tested methods on the three datasets for various values of s_a (default $s_a = 0.001$ for Taxi, $s_a = 0.01$ for MTR and Flights) and s_r (default $s_r = 0.5$). Observe that the optimizations pay off, since the initial cost of the unoptimized Algorithm 1 drops by about 50% when all optimizations are applied for the default s_a and s_r values. The optimizations have larger effect for larger values of s_a, since there number of patterns increase with s_a and so do the savings by avoiding recounting and checking. The savings by the prefix sum optimization are not impressive, because the other optimizations already reduce a lot the number of candidates for which exact counting is required. The second column of Figure 5 breaks down the cost the default values of s_a and s_r; we show the fraction of the

[5] https://github.com/kosyfakics/SSTD-25
[6] https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
[7] https://www.mtr.com.hk/en/customer/main/index.html
[8] https://www.kaggle.com/datasets/usdot/flight-delays?select=flights.csv

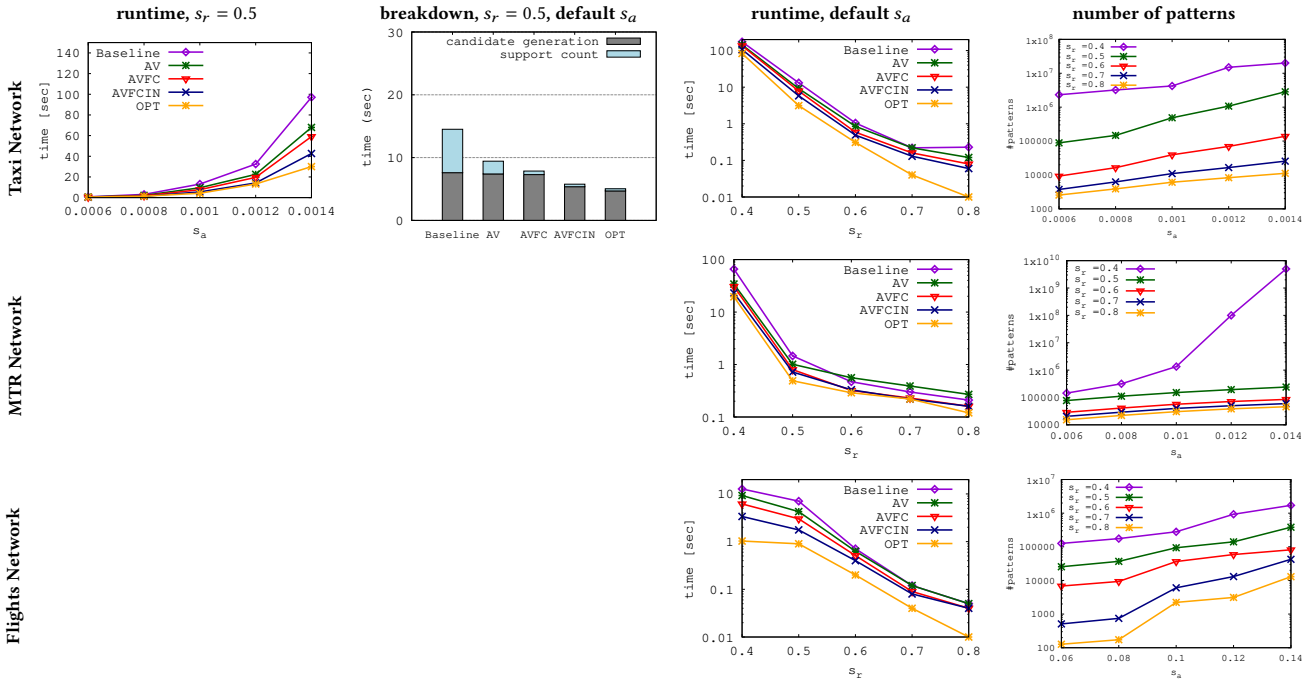

Figure 5: Pattern enumeration experiments

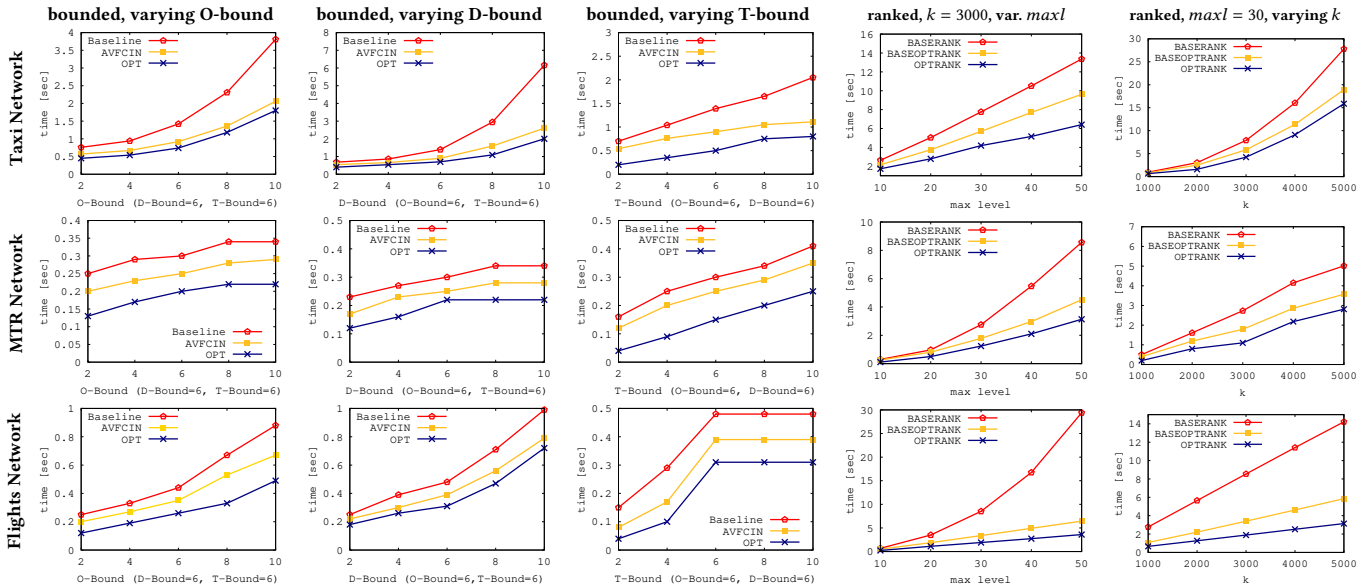

Figure 6: Bounded and ranked-based patterns enumeration cost

cost that goes to candidate pattern generation and support counting. Note that the baseline approach spends most of the time in pattern counting, as the candidate generation process is quite simple. On the other hand, the optimized versions of the algorithm trade off time for pattern generation (spent on bookkeeping all generated triples at each level, bookkeeping OD pairs with at least one trip, etc.) to reduce the time spent on support counting. Note that the ratio of the time spent on support counting is eventually minimized.

When comparing between the different versions, we observe that the candidate generation time drops as more optimizations are employed (e.g., fast check for zero support). Since finding all patterns at level ℓ requires considering all possible extensions of patterns at level $\ell - 1$, we note that there is little room for further reducing the cost of ODT pattern enumeration; in this respect OPT is the best approach that one can apply if the goal is to find all ODT patterns. The pattern enumeration cost increases for smaller values of s_r,

since reducing s_r greatly increases the number of patterns (see the last column of the figure). The high cost of pattern enumeration stems from the fact that a very large number of patterns are found at each level, which, in turn, all have to be minimally generalized due to the weak monotonicity property of Definition 3.9. As the number of patterns grow, so does the essential cost of candidate generation, which becomes the dominant cost factor.

Memory Requirements Table 3 shows the runtime performance (sec), the number of patterns, and the memory requirements (GB) of all methods for different values of s_a on the Taxi dataset. Observe that the baseline algorithm requires significantly less memory compared to the optimized methods, as it does not bookkeep any information (counted triples at each level, $r.dests$ and $r.srcs$ for each atomic region r, prefix-sums, etc.) Still, the additional memory requirements by the optimized methods are bearable and come with significant cost savings, as we have discussed already. In addition, large s_a values that have increased memory requirements result in a huge number of patterns, which may not be informative.

Table 3: Peak memory used, varying s_a - Taxi Network

s_a	Performance	Baseline	AV	AVFC	AVFCIN	OPT
0.001	#patterns	485K	485K	485K	485K	485K
	Memory (GB)	0.38	0.83	0.95	0.97	0.99
	Time (sec)	13.1	9.5	7.6	5.5	4.9
0.0012	#patterns	1.07M	1.07M	1.07M	1.07M	1.07M
	Memory (GB)	0.79	1.42	1.61	1.92	2.1
	Time (sec)	32.46	22.52	19.5	14	13.2
0.0014	#patterns	2.8M	2.8M	2.8M	2.8M	2.8M
	Memory (GB)	1.1	2.72	2.88	2.99	3.4
	Time (sec)	97.7	67.9	59.8	42.7	30.3
0.0016	#patterns	23.6M	23.6M	23.6M	23.6M	23.6M
	Memory (GB)	5.6	7.02	7.28	7.69	8.04
	Time (sec)	529.4	288.9	261.1	242.6	229.8

6.3 Bounded and ranked-based patterns

Bounded patterns As discussed in Section 5.1, one way to limit the number of patterns (and the computational cost) is to bound the number of atomic regions and/or atomic timeslots in them. The first three columns of Figure 6 show the effect of pattern size constraints to the runtime of Baseline, AVFCIN, and OPT. We run experiments by setting s_a and s_r to their default values and fixing the upper bound to the sizes of two of O, D, and T, and vary the bound of one. For instance, in the first column of Figure 6, we keep the upper size bounds of D and T fixed and we vary the upper size bound of O. In general, the cost increases as one bound increases, which is expected, because the number of patterns and generated candidates increases as well. On certain datasets (e.g., MTR), the cost growth is slow when the bound of O or D is increased; this is due to the fact that the number of patterns at low levels is already quite small and the generated patterns start to decrease as we change levels, so the bound increase does not affect the cost significantly. On the other hand, when the bound of T increases, there is a stable increase of time in all datasets. This is due to the fact that the number of atomic timeslots is significantly small and neighboring timeslots are highly correlated in terms of flow. Observe that OPT maintains a significant performance advantage over less optimized versions for different bound values, especially on MTR.

Rank-based patterns The performance of the rank-based pattern enumeration algorithm, described in Section 5.3, is evaluated in the last two columns of Figure 6. We compare three algorithms. The first one is the baseline approach described in Section 5.3.1, without the pattern enumeration optimizations described in Section 4.2. The second one is the baseline approach of Section 5.3.1 with the pattern enumeration optimizations described in Section 4.2. The third approach is the optimized algorithm for rank-based patterns described in Section 5.3.2. The three approaches are denoted by BASERANK, BASEOPTRANK, and OPTRANK, respectively. In all experiments, we set for $s_a = 0.1$. First, we fix $k = 3000$ patterns per level and vary the maximum level $maxl$ of patterns that we generate and enumerate.[9] Observe that OPTRANK maintains a large advantage over the other approaches which do not take advantage of the pruning conditions and the ranking of generated triples. In the last column of the figure, we fix $maxl = 30$ and vary k. Note that the advantage of OPTRANK over the other algorithms is not affected by k. Overall, despite the large value of s_a being used, all algorithms are scalable, making pattern enumeration practical, even in cases where the number of ODT combinations is huge.

6.4 Use case

Lastly, we explore the utility of ODT patterns in real-world applications. To do that, we restricted the origin and time dimensions, according to Section 5.2, and identified the most popular destinations for different times of the day. Table 4 shows some of these patterns in the Taxi Network dataset, indicating that people tend to move toward residential areas or popular dining spots in the afternoon, and a preference for destinations near their homes by the end of the day. An interesting observation is the shift in movement across districts during lunch, possibly due to limited dining options in the Financial District. Insights like these could help restaurant owners identify good locations for new establishments, considering the demand in these areas.

Table 4: Use case - Taxi Network

Origin	Timeslots	popular destinations
FinancialDistrict	[8:30-9:30]	GreenwichVillageS, GreenwichVillageN
FinancialDistrict	[8:30-9:30]	GreenwichVillageS, GreenwichVillageN, Tribeca
FinancialDistrict	[12:30-14:00]	GreenwichVillageS, GreenwichVillageN, Tribeca
FinancialDistrict	[12:30-14:00]	GreenwichVillageS, GreenwichVillageN, Tribeca, BatteryPark
FinancialDistrict	[17:30-19:30]	MidTownCentre, MidTownEast, MurrayHill
FinancialDistrict	[17:30-19:30]	MidTownCentre, MidTownEast, MurrayHill, Union Square, StuyTown

7 CONCLUSIONS

We have studied the problem of enumeration origin-destination-timeslot (ODT) patterns at varying granularity from a database of trips. To our knowledge, this is the first work that formulates and studies this problem. We explore the problem space level-by-level, building on a weak monotonicity property of patterns. We propose a number of optimizations that greatly reduce the cost of pattern enumeration. To address the possibly huge number of ODT patterns, which take too long to enumerate and analyze, we propose and optimize practical variants apply in a smaller search space. Experiments with three real datasets demonstrate the efficiency of the proposed techniques. In the future, we plan to approximate and sampling approaches of reduced time and space complexity.

[9] Recall that the top-k patterns selected per level may generate numerous triples at the next level and there is no s_r threshold to reduce them, so the number of levels can become too large. We use $maxl$ as a parameter for limiting the sizes of patterns.

REFERENCES

[1] 2008. Association Rules, Spatio-temporal. In *Encyclopedia of GIS*. Springer, 32.

[2] Rakesh Agrawal, Tomasz Imielinski, and Arun N. Swami. 1993. Mining Association Rules between Sets of Items in Large Databases. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, Washington, DC, USA, May 26-28*. 207–216.

[3] Rakesh Agrawal, Heikki Mannila, Ramakrishnan Srikant, Hannu Toivonen, and A. Inkeri Verkamo. 1996. Fast Discovery of Association Rules. In *Advances in Knowledge Discovery and Data Mining*. AAAI/MIT Press, 307–328.

[4] Rakesh Agrawal and Ramakrishnan Srikant. 1995. Mining Sequential Patterns. In *Proceedings of the Eleventh International Conference on Data Engineering, March 6-10, Taipei, Taiwan*. IEEE Computer Society, 3–14.

[5] Luis Otávio Alvares, Vania Bogorny, Bart Kuijpers, José Antônio Fernandes de Macêdo, Bart Moelans, and Alejandro A. Vaisman. 2007. A model for enriching trajectories with semantic geographical information. In *15th ACM International Symposium on Geographic Information Systems, ACM-GIS, November 7-9, Seattle, Washington, USA, Proceedings*. ACM, 22.

[6] Mohd Yousuf Ansari, Amir Ahmad, Shehroz S. Khan, Gopal Bhushan, and Mainuddin. 2020. Spatiotemporal clustering: a review. *Artif. Intell. Rev.* 53, 4 (2020), 2381–2423.

[7] Kalidas Ashok. 1996. *Estimation and prediction of time-dependent origin-destination flows*. Ph. D. Dissertation. Massachusetts Institute of Technology.

[8] Muhammad Tayyab Asif, Justin Dauwels, Chong Yang Goh, Ali Oran, Esmail Fathi, Muye Xu, Menoth Mohan Dhanya, Nikola Mitrovic, and Patrick Jaillet. 2014. Spatiotemporal Patterns in Large-Scale Traffic Speed Prediction. *IEEE Trans. Intell. Transp. Syst.* 2 (2014), 794–804.

[9] Jaume Barceló, Xavier Ros-Roca, and Lidia Montero. 2022. Data Analytics and Models for Understanding and Predicting Travel Patterns in Urban Scenarios. In *The Evolution of Travel Time Information Systems: The Role of Comprehensive Traffic Models and Improvements Towards Cooperative Driving Environments*. Springer, 201–277.

[10] Jasper Beernaerts, Ellen Debever, Matthieu Lenoir, Bernard De Baets, and Nico Van de Weghe. 2019. A method based on the Levenshtein distance metric for the comparison of multiple movement patterns described by matrix sequences of different length. *Expert Syst. Appl.* 115 (2019), 373–385.

[11] KN Behara and Ashish Bhaskar. 2021. Can partial structural information of travel demand improve the quality of OD matrix estimates. In *Proceedings of the Australasian Transport Research Forum, Brisbane, Australia*. 8–10.

[12] Krishna NS Behara, Ashish Bhaskar, and Edward Chung. 2021. A DBSCAN-based framework to mine travel patterns from origin-destination matrices: Proof-of-concept on proxy static OD from Brisbane. *Transportation Research Part C: Emerging Technologies* 131 (2021), 103370.

[13] Krishna NS Behara, Ashish Bhaskar, and Edward Chung. 2022. Single-level approach to estimate origin-destination matrix: exploiting turning proportions and partial OD flows. *Transportation Letters* 14, 7 (2022), 721–732.

[14] Krishna N. S. Behara, Ashish Bhaskar, and Edward Chung. 2022. Geographical window based structural similarity index for origin-destination matrices comparison. *J. Intell. Transp. Syst.* 26, 1 (2022), 46–67.

[15] Jiannan Cai and Mei-Po Kwan. 2022. Discovering co-location patterns in multivariate spatial flow data. *Int. J. Geogr. Inf. Sci.* 36, 4 (2022), 720–748.

[16] Yuhan Cai and Raymond T. Ng. 2004. Indexing Spatio-Temporal Trajectories with Chebyshev Polynomials. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, Paris, France, June 13-18*. ACM, 599–610.

[17] Huiping Cao, Nikos Mamoulis, and David W. Cheung. 2005. Mining Frequent Spatio-Temporal Sequential Patterns. In *Proceedings of the 5th IEEE International Conference on Data Mining (ICDM), 27-30 November, Houston, Texas, USA*. IEEE Computer Society, 82–89.

[18] Tingyang Chen, Lugang Nie, Jiwei Pan, Lai Tu, Bolong Zheng, and Xiang Bai. 2022. Origin-Destination Traffic Prediction based on Hybrid Spatio-Temporal Network. In *IEEE International Conference on Data Mining, ICDM, Orlando, FL, USA, November 28 - Dec. 1*. IEEE, 879–884.

[19] Dong-Wan Choi, Jian Pei, and Thomas Heinis. 2017. Efficient Mining of Regional Movement Patterns in Semantic Trajectories. *Proc. VLDB Endow.* 10, 13 (2017), 2073–2084.

[20] Kai-Fung Chu, Albert YS Lam, and Victor OK Li. 2019. Deep multi-scale convolutional LSTM network for travel demand and origin-destination predictions. *IEEE Transactions on Intelligent Transportation Systems* 21, 8 (2019), 3219–3232.

[21] Tamara Djukic, Serge Hoogendoorn, and Hans Van Lint. 2013. *Reliability assessment of dynamic OD estimation methods based on structural similarity index*. Technical Report.

[22] Bowen Du, Hao Peng, Senzhang Wang, Md. Zakirul Alam Bhuiyan, Lihong Wang, Qiran Gong, Lin Liu, and Jing Li. 2020. Deep Irregular Convolutional Residual LSTM for Urban Traffic Passenger Flows Prediction. *IEEE Trans. Intell. Transp. Syst.* 21, 3 (2020), 972–985.

[23] Zongtao Duan, Kai Zhang, Zhe Chen, Zhiyuan Liu, Lei Tang, Yun Yang, and Yuanyuan Ni. 2019. Prediction of city-scale dynamic taxi origin-destination flows using a hybrid deep neural network combined with travel time. *IEEE Access* 7 (2019), 127816–127832.

[24] Qi Fan, Dongxiang Zhang, Huayu Wu, and Kian-Lee Tan. 2016. A General and Parallel Platform for Mining Co-Movement Patterns over Large-scale Trajectories. *Proc. VLDB Endow.* (2016), 313–324.

[25] Zheng Fang, Qingqing Long, Guojie Song, and Kunqing Xie. 2021. Spatial-Temporal Graph ODE Networks for Traffic Flow Forecasting. In *KDD: The 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, Singapore, August 14-18*. ACM, 364–373.

[26] Fosca Giannotti, Mirco Nanni, and Dino Pedreschi. 2006. Efficient Mining of Temporally Annotated Sequences. In *Proceedings of the Sixth SIAM International Conference on Data Mining, April 20-22, Bethesda, MD, USA*. SIAM, 348–359.

[27] Fosca Giannotti, Mirco Nanni, Fabio Pinelli, and Dino Pedreschi. 2007. Trajectory pattern mining. In *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Jose, California, USA, August 12-15*. ACM, 330–339.

[28] Yongshun Gong, Zhibin Li, Jian Zhang, Wei Liu, and Yu Zheng. 2022. Online Spatio-Temporal Crowd Flow Distribution Prediction for Complex Metro System. *IEEE Trans. Knowl. Data Eng.* 34, 2 (2022), 865–880.

[29] Joachim Gudmundsson, Patrick Laube, and Thomas Wolle. 2008. Movement Patterns in Spatio-temporal Data. In *Encyclopedia of GIS*. Springer, 726–732.

[30] Marios Hadjieleftheriou, George Kollios, Petko Bakalov, and Vassilis J. Tsotras. 2005. Complex Spatio-Temporal Pattern Queries. In *Proceedings of the 31st International Conference on Very Large Data Bases, Trondheim, Norway, August 30 - September 2*. ACM, 877–888.

[31] Jiawei Han and Yongjian Fu. 1999. Mining Multiple-Level Association Rules in Large Databases. *IEEE Trans. Knowl. Data Eng.* (1999), 798–804.

[32] Jiawei Han, Jian Pei, Yiwen Yin, and Runying Mao. 2004. Mining Frequent Patterns without Candidate Generation: A Frequent-Pattern Tree Approach. *Data Min. Knowl. Discov.* (2004), 53–87.

[33] Martin L Hazelton. 2001. Inference for origin–destination matrices: estimation, prediction and reconstruction. *Transportation Research Part B: Methodological* 35, 7 (2001), 667–676.

[34] Ching-Tien Ho, Rakesh Agrawal, Nimrod Megiddo, and Ramakrishnan Srikant. 1997. Range Queries in OLAP Data Cubes. In *SIGMOD, Proceedings ACM SIGMOD International Conference on Management of Data, May 13-15,Tucson, Arizona, USA*. ACM Press, 73–88.

[35] Ruoming Jin, Lin Liu, Bolin Ding, and Haixun Wang. 2011. Distance-Constraint Reachability Computation in Uncertain Graphs. *Proc. VLDB Endow.* 4, 9 (2011), 551–562.

[36] Panos Kalnis, Nikos Mamoulis, and Spiridon Bakiras. 2005. On Discovering Moving Clusters in Spatio-temporal Data. In *Advances in Spatial and Temporal Databases, 9th International Symposium, SSTD, Angra dos Reis, Brazil, August 22-24, Proceedings (Lecture Notes in Computer Science, Vol. 3633)*. Springer, 364–381.

[37] Juyoung Kang and Hwan-Seung Yong. 2010. Mining Spatio-Temporal Patterns in Trajectory Data. *J. Inf. Process. Syst.* 6, 4 (2010), 521–536.

[38] Krzysztof Koperski and Jiawei Han. 1995. Discovery of Spatial Association Rules in Geographic Information Databases. In *Advances in Spatial Databases, 4th International Symposium, SSD, Portland, Maine, USA, August 6-9, Proceedings (Lecture Notes in Computer Science)*. Springer, 47–66.

[39] Chrysanthi Kosyfaki, Nikos Mamoulis, Evaggelia Pitoura, and Panayiotis Tsaparas. 2019. Flow Motifs in Interaction Networks. In *Advances in Database Technology - 22nd International Conference on Extending Database Technology, EDBT, Lisbon, Portugal, March 26-29*. 241–252.

[40] Chrysanthi Kosyfaki, Nikos Mamoulis, Evaggelia Pitoura, and Panayiotis Tsaparas. 2021. Flow Computation in Temporal Interaction Networks. In *37th IEEE International Conference on Data Engineering, ICDE, Chania, Greece, April 19-22*. IEEE, 660–671.

[41] Sakitha Kumarage, Mehmet Yildirimoglu, and Zuduo Zheng. 2023. A hybrid modelling framework for the estimation of dynamic origin–destination flows. *Transportation Research Part B: Methodological* 176 (2023).

[42] Antonino La Barbera and Bernardo Spagnolo. 2002. Spatio-temporal patterns in population dynamics. *Physica A: Statistical Mechanics and its Applications* 314, 1-4 (2002), 120–124.

[43] Patrick Laube, Mark de Berg, and Marc J. van Kreveld. 2008. Spatial Support and Spatial Confidence for Spatial Association Rules. In *Headway in Spatial Data Handling, 13th International Symposium on Spatial Data Handling, Montpellier, France, 23-25 July (Lecture Notes in Geoinformation and Cartography)*. Springer, 575–593.

[44] Yan Lin, Huaiyu Wan, Jilin Hu, Shengnan Guo, Bin Yang, Youfang Lin, and Christian S Jensen. 2023. Origin-Destination Travel Time Oracle for Map-based Services. *arXiv preprint arXiv:2307.03048* (2023).

[45] Jia Liu, Tianrui Li, Shenggong Ji, Peng Xie, Shengdong Du, Fei Teng, and Junbo Zhang. 2023. Urban Flow Pattern Mining Based on Multi-Source Heterogeneous Data Fusion and Knowledge Graph Embedding. *IEEE Trans. Knowl. Data Eng.* 35, 2 (2023), 2133–2146.

[46] Jeremy L. Mennis and Jun Wei Liu. 2005. Mining Association Rules in Spatio-Temporal Data: An Analysis of Urban Socioeconomic and Land Cover Change. *Trans. GIS* (2005), 5–17.

[47] Yasuhiko Morimoto. 2001. Mining frequent neighboring class sets in spatial databases. In *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining, San Francisco, CA, USA, August 26-29*. ACM, 353–358.

[48] Amit Anil Nanavati, Krishna Prasad Chitrapura, Sachindra Joshi, and Raghu Krishnapuram. 2001. Mining Generalised Disjunctive Association Rules. In *Proceedings of the ACM CIKM International Conference on Information and Knowledge Management, Atlanta, Georgia, USA, November 5-10*. ACM, 482–489.

[49] Teresa Pamula and Renata Zochowska. 2023. Estimation and prediction of the OD matrix in uncongested urban road network based on traffic flows using deep learning. *Eng. Appl. Artif. Intell.* 117, Part (2023), 105550.

[50] Ashwin Paranjape, Austin R. Benson, and Jure Leskovec. 2017. Motifs in Temporal Networks. In *Proceedings of the Tenth ACM International Conference on Web Search and Data Mining, WSDM, Cambridge, United Kingdom, February 6-10*. ACM, 601–610.

[51] Ram M Pendyala and Konstadinos G Goulias. 2002. Time use and activity perspectives in travel behavior research. *Transportation* 29, 1 (2002), 1–4.

[52] Can Rong, Zhicheng Liu, Jingtao Ding, and Yong Li. 2024. Learning to Generate Temporal Origin-destination Flow Based-on Urban Regional Features and Traffic Information. *ACM Trans. Knowl. Discov. Data* 18, 6 (2024), 150:1–150:17.

[53] Xavier Ros-Roca, Lídia Montero, Jaume Barceló, Klaus Nökel, and Guido Gentile. 2022. A practical approach to assignment-free dynamic origin–destination matrix estimation problem. *Transportation Research Part C: Emerging Technologies* 134 (2022).

[54] Hongzhi Shi, Quanming Yao, Qi Guo, Yaguang Li, Lingyu Zhang, Jieping Ye, Yong Li, and Yan Liu. 2020. Predicting Origin-Destination Flow via Multi-Perspective Graph Convolutional Network. In *36th IEEE International Conference on Data Engineering, ICDE, Dallas, TX, USA, April 20-24*. IEEE, 1818–1821.

[55] Pang-Ning Tan, Michael Steinbach, Vipin Kumar, Christopher Potter, Steven Klooster, and Alicia Torregrosa. 2001. Finding spatio-temporal patterns in earth science data. In *KDD Workshop on Temporal Data Mining*, Vol. 19.

[56] Yufei Tao, Christos Faloutsos, Dimitris Papadias, and Bin Liu. 2004. Prediction and Indexing of Moving Objects with Unknown Motion Patterns. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, Paris, France, June 13-18*. ACM, 611–622.

[57] Ping Tu, Wei Yao, Zhiyuan Zhao, Pengzhou Wang, Sheng Wu, and Zhixiang Fang. 2022. Interday stability of taxi travel flow in urban areas. *ISPRS International Journal of Geo-Information* 11, 12 (2022), 590.

[58] Junmei Wang, Wynne Hsu, and Mong-Li Lee. 2005. A framework for mining topological patterns in spatio-temporal databases. In *Proceedings of the ACM CIKM International Conference on Information and Knowledge Management, Bremen, Germany, October 31 - November 5,*. ACM, 429–436.

[59] Yuandong Wang, Hongzhi Yin, Hongxu Chen, Tianyu Wo, Jie Xu, and Kai Zheng. 2019. Origin-Destination Matrix Prediction via Graph Convolution: a New Perspective of Passenger Demand Modeling. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD, Anchorage, AK, USA, August 4-8*. ACM, 1227–1235.

[60] Dawen Xia, Shunying Jiang, Nan Yang, Yang Hu, Yantao Li, Huaqing Li, and Lin Wang. 2021. Discovering spatiotemporal characteristics of passenger travel with mobile trajectory big data. *Physica A: Statistical Mechanics and Its Applications* 578 (2021), 126056.

[61] Xiaochen Xian, Honghan Ye, Xin Wang, and Kaibo Liu. 2021. Spatiotemporal Modeling and Real-Time Prediction of Origin-Destination Traffic Demand. *Technometrics* 63, 1 (2021), 77–89.

[62] Xiaochen Xian, Honghan Ye, Xin Wang, and Kaibo Liu. 2021. Spatiotemporal modeling and real-time prediction of origin-destination traffic demand. *Technometrics* 63, 1 (2021), 77–89.

[63] Peng Xie, Tianrui Li, Jia Liu, Shengdong Du, Xin Yang, and Junbo Zhang. 2020. Urban flow prediction from spatiotemporal data using machine learning: A survey. *Information Fusion* 59 (2020), 1–12.

[64] Jiexia Ye, Juanjuan Zhao, Liutao Zhang, Cheng-Zhong Xu, Jun Zhang, and Kejiang Ye. 2020. A Data-Driven Method for Dynamic OD Passenger Flow Matrix Estimation in Urban Metro Systems. In *Big Data - BigData - 9th International Conference, Held as Part of the Services Conference Federation, SCF, Honolulu, HI, USA, September 18-20, Proceedings (Lecture Notes in Computer Science, Vol. 12402)*. Springer, 116–126.

[65] Jin Soung Yoo and Mark Bow. 2011. Mining top-k closed co-location patterns. In *IEEE International Conference on Spatial Data Mining and Geographical Knowledge Services, ICSDM, Fuzhou, China, June 29 - July 1*. IEEE, 100–105.

[66] Piao Yu, Xu Zhang, Yongshun Gong, Jian Zhang, Haoliang Sun, Junjie Zhang, Xinxin Zhang, and Yilong Yin. 2025. Enhancing origin-destination flow prediction via bi-directional spatio-temporal inference and interconnected feature evolution. *Expert Syst. Appl.* 264 (2025), 125679.

[67] Qinyue Yu, Yuli Gu, Siyuan Yang, and Mengjie Zhou. 2021. Discovering spatiotemporal patterns and urban facilities determinants of cycling activities in Beijing. *Journal of Geovisualization and Spatial Analysis* 5, 1 (2021), 16.

[68] Wenhao Yu. 2016. Spatial co-location pattern mining for location-based services in road networks. *Expert Syst. Appl.* (2016), 324–335.

[69] Haitao Yuan, Guoliang Li, and Zhifeng Bao. 2022. Route Travel Time Estimation on A Road Network Revisited: Heterogeneity, Proximity, Periodicity and Dynamicity. *Proc. VLDB Endow.* 16, 3 (2022), 393–405.

[70] Haitao Yuan, Guoliang Li, Zhifeng Bao, and Ling Feng. 2020. Effective Travel Time Estimation: When Historical Trajectories over Road Networks Matter. In *Proceedings of the International Conference on Management of Data, SIGMOD Conference, online conference [Portland, OR, USA], June 14-19*. ACM, 2135–2149.

[71] Kuan-Hsuan Yung, Josh Jia-Ching Ying, Hui-Ting Lin, and Vincent S. Tseng. 2024. Learning Location Semantics and Dynamics for Traffic Origin-Destination Demand Prediction. In *International Joint Conference on Neural Networks, IJCNN, Yokohama, Japan, June 30 - July 5*. IEEE, 1–8.

[72] Shahriar Afandizadeh Zargari, Amirmasoud Memarnejad, and Hamid Mirzahossein. 2021. Hourly Origin-Destination Matrix Estimation Using Intelligent Transportation Systems Data and Deep Learning. *Sensors* 21, 21 (2021), 7080.

[73] Junbo Zhang, Yu Zheng, Dekang Qi, Ruiyuan Li, Xiuwen Yi, and Tianrui Li. 2018. Predicting citywide crowd flows using deep spatio-temporal residual networks. *Artificial Intelligence* 259 (2018), 147–166.

[74] Xin Zhang, Nikos Mamoulis, David W. Cheung, and Yutao Shou. 2004. Fast mining of spatial collocations. In *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Seattle, Washington, USA, August 22-25*. ACM, 384–393.

[75] Xiexin Zou, Shiyao Zhang, Chenhan Zhang, JQ James, and Edward Chung. 2021. Long-term origin-destination demand prediction with graph deep learning. *IEEE Transactions on Big Data* 8, 6 (2021), 1481–1495.